Q0002

2026年3月29日 19:15

2. 两数相加

已解答

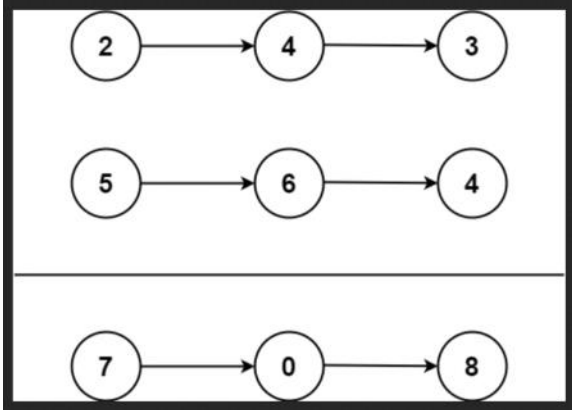
中等 相关标签 相关企业 Aa

给你两个 **非空** 的链表，表示两个非负的整数。它们每位数字都是按照 **逆序** 的方式存储的，并且每个节点只能存储 **一位** 数字。

请你将两个数相加，并以相同形式返回一个表示和的链表。

你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

示例 1：




```
输入：l1 = [2,4,3], l2 = [5,6,4]
输出：[7,0,8]
解释：342 + 465 = 807.
```

示例 2：

```
输入：l1 = [0], l2 = [0]
输出：[0]
```

示例 3：

```
输入：l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
输出：[8,9,9,9,0,0,0,1]
```

提示：

- 每个链表中的节点数在范围 `[1, 100]` 内
- `0 <= Node.val <= 9`
- 题目数据保证列表表示的数字不含前导零

先贴一个我2026.3.29的解法：
因为审题不清 以为l1 l2是顺序的 导致觉得相加进位的方法繁琐 所以诞生了这个无脑解法

```
package Q0002;
import java.math.BigInteger;
class Solution {
    public int length(ListNode l){
        ListNode pointer = l;
        int len=1;
        while(pointer.next!=null){
            pointer=pointer.next;
            len++;
        }
        return len;
    }
```

```java
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode p1=l1;
        ListNode p2=l2;
        BigInteger sum1=new BigInteger("0");
        BigInteger sum2=new BigInteger("0");
        int step1=0;
        int step2=0;
        while(p1!=null){
            BigInteger power = BigInteger.TEN.pow(step1);
            BigInteger term = BigInteger.valueOf(p1.val).multiply(power);
            sum1 = sum1.add(term);
            p1=p1.next;
            step1++;
        }
        while(p2!=null){
            BigInteger power = BigInteger.TEN.pow(step2);
            BigInteger term = BigInteger.valueOf(p2.val).multiply(power);
            sum2 = sum2.add(term);
            p2=p2.next;
            step2++;
        }
        BigInteger sum3=new BigInteger(String.valueOf(sum1.add(sum2)));
        String str= sum3.toString();
        ListNode l3=new ListNode();
        ListNode p3=l3;
        l3.val=str.charAt(str.length()-1)-'0';
        for(int i=1;i<str.length();i++){
            p3.next=new ListNode(str.charAt(str.length()-1-i)-'0');;
            p3=p3.next;
        }
        return l3;
    }
}
```

这里length函数实则也是多余的。。。

做的过程中遇到了这样几个问题：
1.数据类型大小不够（最重要的问题）
2.如何从高位到低位输出一组数的各个位数（这里实则是题目看错而衍生出来的问题）

解决问题：
先从2开始：

```java
String str= sum3.toString();
ListNode l3=new ListNode();
ListNode p3=l3;
l3.val=str.charAt(0)-'0';
for(int i=1;i<str.length();i++){
  p3.next=new ListNode(str.charAt(i)-'0');;
  p3=p3.next;
}
```

我们用toString函数将数字转化为字符串 用字符串可以按顺序查找字符的特性解决了这一问题
（题目看错了 所以跟答案是反着来的）

回到1：
这个题的最大样例达到了20位数 超过了int以及long范围 使得我这个不巧妙的做法被迫使用了BIgInteger类 接下

来先看一下这个类的基本使用情况：

来自 <https://chat.deepseek.com/a/chat/s/21553416-5324-4d41-a81b-aa5970101883>

```java
import java.math.BigInteger;
public class BigIntegerDemo {
    public static void main(String[] args) {
        // 构造
        BigInteger a = new BigInteger("12345678901234567890");
        BigInteger b = BigInteger.valueOf(9876543210L);
        // 算术
        System.out.println("a + b = " + a.add(b));
        System.out.println("a * b = " + a.multiply(b));
        System.out.println("a / b = " + a.divide(b));          // 整数除法
        System.out.println("a mod b = " + a.mod(b));           // 余数非负
        System.out.println("2^100 = " + BigInteger.TWO.pow(100));
        // 比较
        if (a.compareTo(b) > 0) {
            System.out.println("a > b");
        }
        // 位运算
        BigInteger x = new BigInteger("1010", 2);  // 二进制 1010 = 10
        BigInteger y = new BigInteger("1100", 2);  // 12
        System.out.println("x & y = " + x.and(y)); // 1000 (8)
        // 类型转换
        System.out.println("a 的十进制字符串: " + a.toString());
        System.out.println("a 的十六进制: " + a.toString(16));
        long longVal = a.longValueExact();  // 可能溢出，这里会抛异常（因为 a > Long.MAX_VALUE)
    }
}
```

看几点重要的：

1.构造方法

BigInteger(String val)	将十进制字符串转换为 BigInteger
BigInteger.valueOf(long val)	将 long 转换为 BigInteger

2.可以通过静态常量获取常用值：

BigInteger.ZERO、BigInteger.ONE、BigInteger.TWO、BigInteger.TEN

3.常见运算（返回新 BigInteger，不可变）：

add(BigInteger val)	加法
subtract(BigInteger val)	减法
multiply(BigInteger val)	乘法
divide(BigInteger val)	除法（整数除法，向零舍入）
pow(int exponent)	幂运算（exponent 为非负整数）
compareTo(BigInteger val)	比较大小：返回 -1, 0, 1
equals(Object obj)	判断数值是否相等（不可与 int 直接比较）
min(BigInteger val)	返回较小的那个
max(BigInteger val)	返回较大的那个

大致如此。

接下来就是对这个笨重方法的优化了：

```
package Q0002;
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode current = dummy;
        int carry = 0;

        while (l1 != null || l2 != null || carry != 0) {
            int val1 = (l1 != null) ? l1.val : 0;
            int val2 = (l2 != null) ? l2.val : 0;

            int sum = val1 + val2 + carry;
            carry = sum / 10;

            current.next = new ListNode(sum % 10);
            current = current.next;

            if (l1 != null) l1 = l1.next;
            if (l2 != null) l2 = l2.next;
        }

        return dummy.next;
    }
}
```

这里有几个小细节：

1.可以直接拿l1遍历链表 不必再设一个“指针”

2.对链表为null的处理 取0

3.其实可以设头节点 返回头节点.next 这样就避免了第一个要分开讨论的情况